

Real-Time Predictive Crowd Twin Platform by Efficient Data Assimilation

Atsuhiko Yano*, Tatsuya Amano*[†], Hirozumi Yamaguchi*[†]

* Graduate School of Information Science and Technology, The University of Osaka
Suita, Osaka, Japan

[†] RIKEN Center for Computational Science (R-CCS), RIKEN
Kobe, Hyogo, Japan

Abstract—Real-time crowd digital twins require both fast future prediction and continuous synchronization with real-world observations. However, in agent-based simulations that run ahead of real time, newly arriving sensor data correspond to past simulation states, making full rollback and recomputation computationally infeasible under real-time constraints. To address this challenge, this paper proposes a dual-simulation framework called Lead-Chase for indoor crowd digital twins. In the proposed architecture, the Lead simulation continuously advances virtual time faster than real time to provide future predictions, while the Chase simulation selectively corrects discrepancies using delayed observation data. Instead of recomputing all agents, Chase re-simulates only agents with large prediction errors and a limited number of neighboring agents selected by a regression-based improvement estimation model. This design enables efficient data assimilation while maintaining real-time performance. The framework is implemented on Unity using a NavMesh-based mobility model and an XGBoost destination predictor trained from indoor trajectory data. Experiments were conducted using a digital twin of a university campus floor reconstructed in Unity with LiDAR-based tracking data. Results show that the proposed method consistently reduces positioning errors compared to the standalone Lead prediction, achieving more than 2.5 m improvement in mean absolute error across multiple prediction horizons. In addition, we evaluate area-level population distributions by partitioning the floor into functional areas and computing the error between the predicted and ground-truth population counts in each area, confirming that Chase improves the reproduction of spatial crowd distributions.

Index Terms—Crowd digital twin, Real-time prediction, Data assimilation, Agent-based simulation

I. INTRODUCTION

Effective indoor crowd management is crucial for safety and efficiency because it enables administrators to anticipate congestion and guide pedestrians during emergencies. To support these tasks, recent advancements in tracking technologies using LiDAR and Wi-Fi [1] allow systems to monitor individual positions in real time. Yet, monitoring the current state alone is insufficient for proactive control because systems must predict future crowd states to detect potential risks before issues arise. Therefore, integrating real-time sensing with predictive simulation has become a primary objective in this field.

Agent-based simulations are essential for reproducing complex crowd behaviors in structured environments. These models, whether driven by physics-based rules or learned patterns, generate future states recursively to emulate the crowd's evolution over time. However, a fundamental challenge persists in

this continuous operation. A simulation running autonomously in parallel with the real world inevitably drifts from reality due to the accumulation of small errors and unpredictable human actions. Even highly accurate models cannot perfectly anticipate every instantaneous change in the physical world. Consequently, data assimilation techniques have gained attention for their ability to correct the simulation state sequentially using incoming sensor data, ensuring the digital twin remains synchronized with reality.

A fundamental conflict arises when applying data assimilation to real-time prediction. Since the simulation must proceed faster than reality to forecast future states, the logic has already advanced beyond the timestamp of any newly arrived observation. Correcting the state based on this delayed ground truth requires rolling back the simulation and recomputing the subsequent steps. Performing this rollback for the entire crowd is computationally expensive and often exceeds the sensor update interval, which prevents the system from maintaining real-time performance.

To address this computational bottleneck, we propose a dual simulation framework called Lead-Chase that separates the prediction task from the assimilation process. The Lead simulation continuously advances virtual time ahead of real time to forecast future states without interruption, while the Chase simulation operates in the background to assimilate delayed observational data. When new data arrives, Chase quickly recomputes the past trajectories to correct discrepancies. This process selectively updates only the agents with large errors and those influenced by them rather than the entire crowd, allowing the system to maintain accuracy and real-time performance simultaneously.

The effectiveness of this framework relies on accurately identifying which agents require correction. Simply selecting agents with large position errors is insufficient because local changes affect neighboring agents through complex interactions. To solve this, we introduce a regression model trained on offline simulation results to estimate the potential improvement in prediction accuracy for each agent. During online operation, the system selects the top candidates that maximize this expected improvement. Furthermore, we utilize a navigation mesh based on the building geometry to manage the movement of these agents, ensuring that the simulation maintains realistic collision avoidance even during the high-

speed correction process.

We validated the proposed framework using a digital twin environment constructed on the Unity engine that replicates one floor of a campus building. Using real pedestrian tracking data to evaluate prediction accuracy, the results show that the Chase simulation consistently reduces position errors compared to the standalone Lead prediction. Specifically, the proposed method improved the average displacement error by approximately 2.5 meters. In addition, the regression-based selection strategy outperformed random selection, confirming the effectiveness of targeted correction for real-time crowd digital twins.

II. RELATED WORK

A. Crowd Tracking System

In this paper, we assume a crowd tracking system for indoor environments that senses human movement and outputs it as coordinate and velocity data. In particular, in recent years, systems have been developed to observe people's indoor positions using information obtained from vision-independent wireless sensing and environment-side sensors (e.g., cameras and LiDAR) [2], [3]. As a representative example of wireless sensing, WiTrack [4] has been proposed, which tracks three-dimensional human motion without requiring users to carry devices by exploiting reflections from the human body and demonstrates centimeter-level tracking accuracy even through walls or under occlusions. CrowdMirage [1] extracts movement trajectories from Wi-Fi connection logs and enables cross-granularity prediction and visualization.

Meanwhile, from the perspective of large-scale deployment for real-world operation, MLoc [5] has been reported to track smartphone positions in complex mall environments using fingerprint information such as BLE and geomagnetic signals. MLoc includes a design that scales up the collection of training data and an organized set of challenges identified based on operational experience as a real-world service, thereby concretizing the key components of practical crowd tracking. As an example of tracking based on environment-side sensors, a person-tracking pipeline has been reported that combines person detection using 2D LiDAR with lightweight multi-object tracking (MOT) and achieves 20 Hz operation on embedded computing resources, providing a useful reference for designs that assume real-time processing [6]. In addition, Scene Transformer-type models that integrate map information with agent trajectories [7] make digital-twin crowd representations more realistic by being able to estimate diverse future trajectories simultaneously.

As described above, crowd tracking has been advancing along multiple lines, including device-free and occlusion-robust approaches based on wireless sensing, location estimation at a real operational scale using BLE and related signals, and direct observation and high-speed tracking using environment-side sensors such as LiDAR. However, real-time synchronization between the physical and virtual spaces, as well as immediate feedback of prediction results to the real world, is still considered to have a low level of maturity.

Many prior studies focus primarily on visualizing observations and estimating states, and they lack a sufficient mechanism to sequentially assimilate the virtual crowd state upon each arrival of observational data and dynamically correct future predictions [8].

B. Crowd Modeling Methods

Recent studies on modeling crowd behavior have actively explored both data-driven trajectory prediction and simulation-based crowd reproduction. Representative data-driven approaches include Social-LSTM [9] and Scene Transformer [10], which have been shown to estimate future trajectories accurately from past observations. These methods have brought significant progress by learning pedestrian interactions and environmental context, enabling a statistical reproduction of complex crowd behaviors. In particular, on outdoor pedestrian datasets such as ETH and UCY, they achieve high accuracy in short-term prediction, with an average error of less than 1 m. However, these approaches typically assume offline training on large-scale datasets and thus depend strongly on the environmental conditions seen during training. Therefore, in indoor environments where sensor placements and walking routes change frequently, their adaptability to new situations and generalization performance is limited. Moreover, these neural network models depend heavily on the training data distribution and are not designed to update the model sequentially in response to incoming observational data.

In contrast, agent-based simulations such as the Social Force Model [11] can reproduce realistic behaviors by accounting for environmental structure and interactions. These methods are based on physical dynamical models and geometric constraints, offering the advantage of a certain degree of generality even in unseen environments. As an extension of ORCA-type methods, RRVO has also been proposed to mitigate issues such as deadlocks caused by complex shapes, and improvements toward more realistic avoidance behavior in crowded environments have been actively explored [12].

These prediction approaches provide limited mechanisms for updating the simulation state by incorporating sequential observations obtained from the real world. That is, once the model starts making predictions, it cannot dynamically correct the ongoing prediction results even if new sensor data become available afterward. As a result, when real-world crowd distributions and movement patterns change over time, the virtual crowd in the simulation gradually diverges from the actual state of the real world. This divergence becomes particularly pronounced in a short time span in indoor environments, where pedestrian inflows, outflows, and route choices can change rapidly, posing a serious challenge for applications such as real-time control and evacuation guidance.

C. Simulation-Observation Data Assimilation Methods

In recent years, crowd flow prediction based on data assimilation—where real-time incoming observations are incorporated into a simulation to sequentially forecast future

states—has attracted increasing attention. The design of assimilation frameworks varies depending on the granularity of observations, the state variables to be updated, the update frequency, and constraints on computational resources.

For example, Clay et al. applied an Unscented Kalman Filter to an agent-based simulation and presented a framework that performs near-real-time updates while assimilating pseudo sequential observations [13]. In [14], the authors proposed Bayesian Behavioural Model Estimation (BBME) to obtain the long prediction lead times required for intervention in live crowd simulations by sequentially estimating the underlying behavioral model based on observations. BBME uses coarse aggregated observations, such as population maps, and performs online estimation using a particle filter whose state includes behavioral model parameters. By updating forward simulations with observations in this way, the study showed that long-term predictions can be stabilized even when behavioral tendencies vary over time.

In [15], the authors presented a framework that combines a pedestrian agent-based model with a particle filter to assimilate real-time observations into a running simulation. They reported that, while online state estimation itself is feasible, the required number of particles increases exponentially as the number of agents grows, making scalability a primary challenge from a computational cost perspective.

In frameworks that combine prediction with selective resimulation, such as the one proposed in this study, it is possible to limit the set of states to be updated, thereby reducing the computational burden of data assimilation while preserving consistency with observations. As a result, this approach can be positioned as a practical compromise to address the challenges.

III. SYSTEM ARCHITECTURE

A. Assumed Tracking System

Fig. 1 shows the overall architecture of the proposed crowd digital twin platform. The crowd digital twin platform proposed in this paper assumes the use of a tracking system that can obtain human mobility data from LiDARs and cameras installed within the target building.

Such tracking systems provide users with pedestrian position data, along with additional information such as timestamps and velocities, based on real-world observed trajectory data obtained in real time at regular intervals. For example, a pedestrian flow sensing system such as [16] can obtain, in real time at fixed time intervals, a set of ordered pairs (person ID, position) at each time step. Each ordered pair is hereafter referred to as an observation, and the set of observations is referred to as an observation set.

Let the sampling interval be Δs (we typically assume $\Delta s \approx 3$ seconds), and denote the time at which an observation set is obtained by the discrete observation index m ($m \geq 0$). Using m and Δs , the observation time is expressed as $t_m = m\Delta s$.

For each observation $y \in Y_m$, we use $y.ID$ and $y.pos$ to denote the person ID and position of observation y , respectively. For $y \in Y_m$ and $y' \in Y_{m+u}$, if $y.ID = y'.ID$, then y and y'

represent observations of the same person (as determined by the observation system).

Hereafter, assuming an environment in which such observation sets can be obtained, we present the concept of our crowd digital twin platform.

B. System Functionality

Let the current time be denoted by t . A user specifies a future time $t' > t$ and a region of interest to the system, and requests the predicted observation set at time t' , as well as each person's trajectory leading up to that observation set. We assume that each request (indexed by k) arrives at the system without delay, and that the latency between issuing a request and its arrival at the system is negligible—i.e., much smaller than the simulation time resolution—and thus can be ignored. Therefore, when request k is issued at time t and arrives at the system, the system starts a crowd prediction simulation from the latest available observation time t_m at time t (with observation index m). Using the observation set Y_m at t_m as the initial configuration and the observation sets up to Y_m , the system runs the simulation forward until the nearest observation time $t_{m'}$ after t' .

C. Predictive Simulation Framework

The goal of this system is not merely to predict aggregate values such as the total number of people in the entire area but to answer fine-grained prediction queries over arbitrary regions and time horizons, such as future population density in a specific area and the future trajectories of individual pedestrians. To realize such predictive simulation, our architecture places an agent-based simulation as a core. In this simulation, agents are placed in the virtual space based on the positions of individuals obtained from the latest observations. Using features computed from the observations obtained up to the latest time step, the system reproduces realistic movements using a pedestrian mobility model, thereby deriving microscopic individual trajectories of people after the latest observation.

For each user prediction request k , the system launches a simulation instance and runs the simulation. We denote this instance by S_k . Although S_k uses the same time axis as the real world as its simulation time, the simulation clock must advance faster than real time; otherwise, the system cannot produce predictions. We denote by X the ratio indicating how far the simulation time can advance within a unit of real-world time.

If simulation instance S_k is launched at R_k and uses the observation time $t_{m(k)}$ as its initial condition, then its initial virtual time is $s_k(R_k) = t_{m(k)}$. Thereafter, when the real time is $t \geq R_k$, the internal virtual time of simulation S_k , denoted by $s_k(t)$, is given by Eq. 1. The simulation time always represents a time ahead of the latest observation time.

$$s_k(t) = t_{m(k)} + X \cdot (t - R_k) \quad (1)$$

When the virtual time reaches the user-specified query time τ (i.e., $s_k(t) \geq \tau$), the system outputs the following

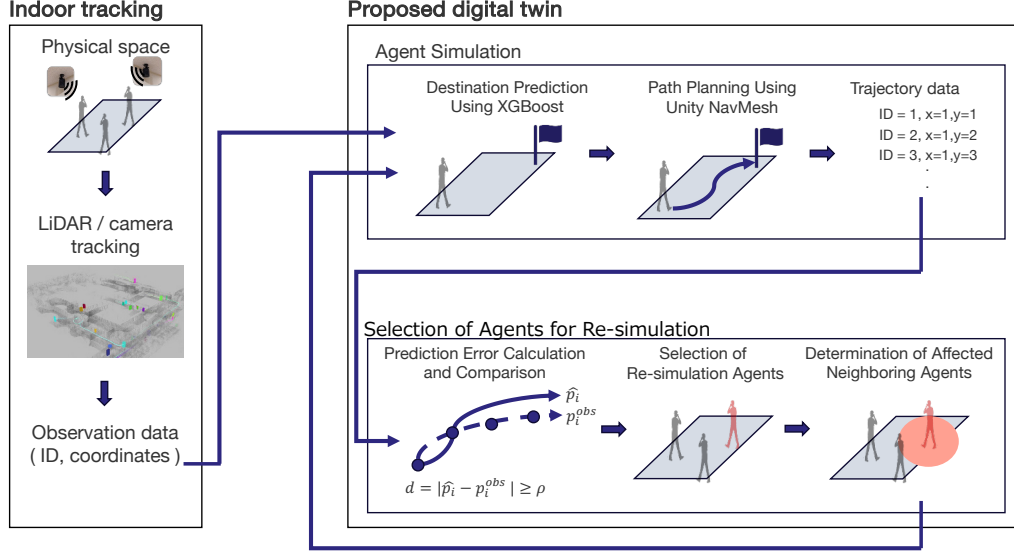


Fig. 1. System overview

information. First, as a field-level quantity, it provides aggregated population-density statistics for each subregion within the target area. Second, as individual trajectories, it outputs a continuous time series of positions for each agent identified by its ID, denoted by $p^i(s)_{s \in [t_{m(k)}, \tau]}$.

To make the agent-based simulation operational, the system must answer two questions: where each person is heading and how they will move. To address these questions, our platform consists of the following two components.

First, the mobility simulation module represents the state of each agent i in the simulation instance S_k by its position $p^{k,i}(s) \in \mathbb{R}^2$ and velocity $v^{k,i}(s) \in \mathbb{R}^2$ at virtual time s . This component functions as a function f that computes the state after a time step of Δs , conditioned on an agent's destination g^i and the states of the other agents.

$$[p^{k,i}(s + \Delta s), v^{k,i}(s + \Delta s)]^T = f(p^{k,i}(s), v^{k,i}(s), g^i, \{p^{k,\ell}(s), v^{k,\ell}(s)\}_{\ell \neq i}; G) \quad (2)$$

f is implemented using Unity's NavMesh and dynamically generates realistic movement behaviors—such as inter-agent collision avoidance and speed adjustment in response to congestion—based on optimal path planning over a NavMesh surface laid on a prebuilt 3D model of the building. By combining NavMesh-based pathfinding with local steering, it simultaneously achieves destination-directed movement and avoidance of interactions with other agents.

Second, the destination estimation module estimates the destination of each agent i , denoted by $g^i \in \mathcal{G}$, online as a probability distribution based on the agent's past movement trajectory. Here, $\mathcal{G}$ denotes the set of candidate destinations. This provides each agent with a dynamic behavioral goal. The notation used throughout this paper is summarized in Table I.

TABLE I
NOTATION TABLE

Symbol	Description
Δs	Observation sampling interval (typically ≈ 3 seconds)
m	Discrete observation index ($m \geq 0$)
t_m	Observation time at index m , $t_m = m\Delta s$
Y_m	Observation set obtained at time t_m
$y \in Y_m$	Single observation at time t_m
$y.ID$	Person ID of observation y
$y.pos$	Position of observation y
t	Current real-world time
t'	User-specified future prediction time
τ	Query time specified by user
k	Index of a user prediction request
S_k	Simulation instance launched for request k
R_k	Real-world launch time of simulation instance S_k
$t_{m(k)}$	Latest observation time used to initialize S_k
$s_k(t)$	Virtual simulation time of instance S_k at real-time t
X	Simulation speed ratio (time acceleration factor)
$p^{k,i}(s)$	Position of agent i in S_k at virtual time s
$v^{k,i}(s)$	Velocity of agent i in S_k at virtual time s
g^i	Destination of agent i
$\mathcal{G}$	Set of candidate destinations
$f(\cdot)$	Mobility update function for agent state transition
G	Environment representation (e.g., NavMesh graph)

IV. LEAD-CHASE FRAMEWORK

A. Challenges of Observation Assimilation in Real-Time Prediction

In the real-time crowd flow prediction system targeted by this study, it is necessary to continuously predict future crowd states using observed pedestrian data from the real world. This system takes as input observation data Y_m ($m = 0, 1, 2, \dots$) obtained at a fixed time interval Δs , where each observation is acquired at real time $t_m = m\Delta s$. The observation Y_m is a set of the positions of all individuals observed at that time. The

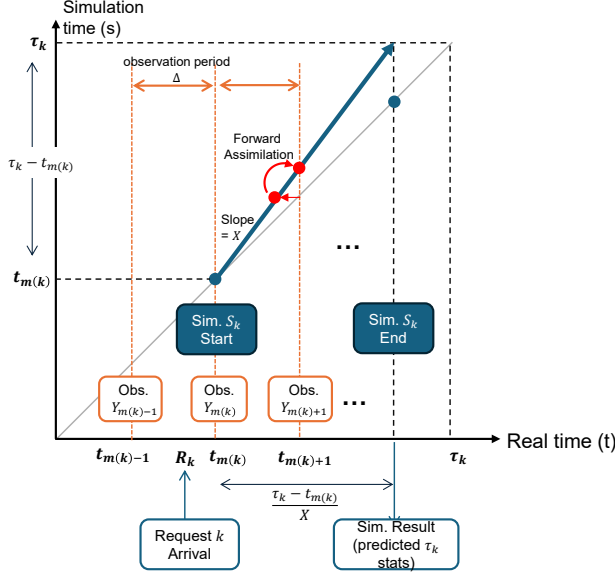


Fig. 2. Problem setting of the Lead-Chase framework

agent-based simulation used to realize prediction maintains a virtual simulation time s , and computes future crowd states by advancing this virtual time as real time elapses. The simulation's progression speed is characterized by a speed-up factor $X > 1$ relative to real time. That is, while Δt elapses in real time, the simulation time advances by $X\Delta t$. This speed-up factor is determined by the simulation time step δ_s and the real time required to compute one step T_{step} , as $X = \delta_s / T_{\text{step}}$. Fig. 2 illustrates the timing relationship between real time, simulation time, observation arrivals, and prediction requests.

We denote the system state at simulation time s as $\mathbf{x}(s)$. The observation Y_m corresponds to a partial observation of this state (positions and velocities), and can be expressed using an observation function h as $Y_m = h(\mathbf{x}^{\text{true}}(t_m)) + \mathbf{v}_m$. Here $\mathbf{x}^{\text{true}}$ denotes the true state in the real world, and $\mathbf{v}_m$ represents the observation error.

Here, an essential difficulty arises. By the time the observation Y_m arrives at real time t_m , the simulation has already progressed to the virtual time $s_m = s_0 + X(t_m - t_0)$. Here s_0 is the initial simulation time, and t_0 is the real time at which the simulation starts. Although the observation Y_m actually represents the real-world state at time t_m , the predicted state at the corresponding simulation time s_m has already been computed. Therefore, even if we attempt to correct the simulation using the error revealed by comparing the observation Y_m with the prediction, the time to be corrected, s_m , is already in the simulation's past. It is not obvious how to revise the ongoing future prediction (i.e., at virtual times $s > s_m$).

A straightforward solution to this problem is to roll back the simulation to time s_m whenever an observation Y_m arrives, and then recompute the simulation using the latest observation as the initial condition. However, this approach is impractical

because advancing the simulation from s_0 to s_m has already taken real time $(t_m - t_0)$. Repeating the same computation would prevent the simulation from catching up before the next observation Y_{m+1} arrives. To maintain real-time performance, the correction computation must be completed within the observation interval Δs , but recomputing at the same speed-up factor makes this impossible.

In the following discussion, for simplicity, we align the initial simulation time with the initial real time and set $s_0 = t_0 = 0$.

To address the above challenge, this study proposes an architecture that combines two simulations with different levels of fidelity: Lead and Chase. Here, fidelity refers to the quality of the simulation, and can be rephrased in terms of its temporal and spatial granularity. Lead advances virtual time ahead of real time, while Chase rapidly corrects past segments based on newly arrived observational data.

$$\mathbf{x} = [\mathbf{p}^1, \mathbf{v}^1, \mathbf{p}^2, \mathbf{v}^2, \dots, \mathbf{p}^N, \mathbf{v}^N]^\top \in \mathbb{R}^{4N} \quad (3)$$

The Lead simulation serves as the primary computation for prediction. Lead is initialized using the observation Y_0 as the initial condition and advances the simulation time with a speed-up factor X_{lead} . The Lead simulation time $s_{\text{lead}}(t)$ at real time t is given by Eq. 4. We also denote the state of Lead as $\mathbf{x}_{\text{lead}}(s)$.

$$s_{\text{lead}}(t) = s_0 + X_{\text{lead}}(t - t_0) \quad (4)$$

When an observation Y_m arrives in real time t_m , Lead has already advanced to the simulation time $s_m = s_0 + X_{\text{lead}}(t_m - t_0)$, and the trajectory $\{\mathbf{x}_{\text{lead}}(s)\}_{s \in [s_0, s_m]}$ has been computed. At this point, Lead must be temporarily paused, and its state must be corrected using the information in the observation Y_m . However, as discussed above, if we recompute from t_m to s_m at the same speed-up factor as Lead, it would require real time $X_{\text{lead}}(s_m - t_m)$, and thus cannot be completed within the observation interval Δs .

Therefore, our framework introduces a fast correction computation using a Chase simulation, which runs at a lower fidelity. Chase operates with a speed-up factor $X_{\text{chase}} > X_{\text{lead}}$ and quickly recomputes the interval from s_0 to s_m using the observation Y_m as the initial condition. The runtime of Chase, T_{chase} is given by Eq. 5.

$$T_{\text{chase}} = \frac{s_m - s_0}{X_{\text{chase}}} = \frac{X_{\text{lead}}(t_m - t_0)}{X_{\text{chase}}} \quad (5)$$

In particular, when $X_{\text{chase}} = X_{\text{lead}}$, we obtain $T_{\text{chase}} = t_m - t_0$, which is equivalent to a full recomputation. However, as discussed above, Chase must finish in a shorter time than Lead.

B. Low-Fidelity Simulation

The essence of the fidelity that determines the speed-up factor X lies in the temporal and spatial granularity of the simulation. Acceleration is achieved by coarsening these granularities. The core contribution of this study lies in advancing the crowd-flow prediction simulation ahead of real time while dynamically maintaining consistency with

observations (ground truth) that become available afterward. That is, Chase is realized by comparing the results of the Lead simulation—run ahead of real time—with the observation data, and re-simulating only the individuals for whom errors are detected. In the following, we describe how to determine which individuals are selected as targets of the Chase simulation.

First, at time t_m , we compute the distance error between the predicted position of each agent i obtained from the Lead simulation, $\hat{\mathbf{p}}_i(t_m)$, and the observed true position, $\mathbf{p}_i^{\text{obs}}(t_m)$, Using Eq. (6)

$$d_i = \|\hat{\mathbf{p}}_i(t_m) - \mathbf{p}_i^{\text{obs}}(t_m)\| \quad (6)$$

At this time, we define the set of agents whose errors exceed the threshold ρ_{th} using Eq. 7.

$$\mathcal{X} = \{i \mid d_i > \rho_{th}\} \quad (7)$$

The set $\mathcal{X}$ consists of agents for which the Lead prediction deviates significantly from the observation, and therefore require correction through re-simulation.

Next, we define a set of agents $\mathcal{Y}$ that may be affected by agents belonging to $\mathcal{X}$. Since the selection of $\mathcal{Y}$ influences future predictions through the execution of Chase, its effect cannot be observed immediately, and it varies nonlinearly due to the re-simulation of $\mathcal{X}$ and the interactions among agents in $\mathcal{Y}$. Therefore, unlike $\mathcal{X}$, it is difficult to formulate $\mathcal{Y}$ in a closed form. To address this, we define the utility of each candidate agent based on the reduction in prediction error after Chase and the additional computation time, and then train a regression model using this utility as the supervisory signal. Concretely, at each observation time, we use as input the features derived from the observation and the Lead prediction, and train a regression model to estimate the expected improvement when each candidate agent i is included in $\mathcal{Y}$. We re-simulate the agents in $\mathcal{X} \cup \mathcal{Y}$ as Chase targets. The state representation, feature design, the definition of gain, and the details of the training procedure are described later.

Finally, we define the set of agents that belong to neither $\mathcal{X}$ nor $\mathcal{Y}$ as $\mathcal{Z}$ according to Eq. 8. Agents in $\mathcal{Z}$ retain the Lead results as is.

$$\mathcal{Z} = \Omega \setminus (\mathcal{X} \cup \mathcal{Y}) \quad (8)$$

The fidelity is controlled by the threshold ρ_{th} . When ρ_{th} is set to a lower value, re-simulation is triggered even by small prediction deviations; however, this increases the computational cost accordingly. In contrast, when ρ_{th} is set to a larger value, a larger error is tolerated, resulting in fewer agents being re-simulated and a shorter execution time; however, the error with respect to the ground truth increases. In other words, execution time and prediction accuracy exhibit a trade-off relationship.

V. MODULE IMPLEMENTATIONS

We describe the implementation of the three modules required to make the simulation operational.

A. Regression Model for Chase Agent Selection

In this study, in addition to the set $\mathcal{X}$ consisting of agents with large observation errors, we select a set of neighboring agents $\mathcal{Y}$ whose future states may change as a result of re-simulating $\mathcal{X}$ and define the Chase target as $\mathcal{X} \cup \mathcal{Y}$. This approach reduces future prediction errors while preventing an increase in computational time by limiting the number of agents to be re-simulated, thereby achieving a balance between prediction accuracy and computational cost. The selection mechanism takes as input features derived from the observation information at each observation time and the Lead predictions. The output of the selection mechanism is the selection of agents to be added to the set $\mathcal{Y}$ from a candidate set excluding $\mathcal{X}$ at each time step. Based on the improvement score $\hat{g}_{t,i}$ estimated by the regression model, the top $(M - |\mathcal{X}|)$ agents are selected to determine $\mathcal{Y}$.

In this study, to enable the learning of the agent selection strategy, Chase is executed offline for all agents. For each agent ID and each observation time t , the observed position $(x_t^{\text{obs}}, y_t^{\text{obs}})$, the position predicted by Lead $(x_t^{\text{lead}}, y_t^{\text{lead}})$, and the position predicted after Chase $(x_t^{\text{chase}}, y_t^{\text{chase}})$ are recorded. Based on these records, the positional errors with respect to the observations are defined as the Euclidean distances

$$e_t^{\text{lead}} = \text{dist}(p_t^{\text{lead}}, p_t^{\text{obs}}), \quad e_t^{\text{chase}} = \text{dist}(p_t^{\text{chase}}, p_t^{\text{obs}})$$

and the improvement achieved by Chase is computed as

$$\text{gain}_t = e_t^{\text{lead}} - e_t^{\text{chase}}$$

The value gain_t is used as a continuous supervisory signal to train a regression model that predicts, for each candidate agent, the expected improvement obtained when that agent is included in Chase. Specifically, we define a function $g_\theta(\cdot)$ that takes as input the feature vector $f_{t,i}$ computed for candidate agent i at time t and outputs the predicted improvement $\hat{g}_{t,i}$

$$\hat{g}_{t,i} = g_\theta(f_{t,i})$$

The model is trained by minimizing the error with respect to the improvement values $\text{gain}_{t,i}$ computed offline. Using the Huber loss, we minimize

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(t,i) \in \mathcal{D}} \text{Huber}(\hat{g}_{t,i} - \text{gain}_{t,i})$$

where $\mathcal{D}$ denotes the training dataset.

During deployment, at each observation update time t , the predicted improvement $\hat{g}_{t,i}$ is estimated for all candidate agents, and the top $M - |\mathcal{X}|$ agents with the largest predicted improvements are selected as the set $\mathcal{Y}$. That is, the Chase set $\mathcal{S}_t$ is defined as

$$\mathcal{S}_t = \mathcal{X} \cup \mathcal{Y}$$

$$\mathcal{Y} = \text{Top-}(M - |\mathcal{X}|)(\{\hat{g}_{t,i}\}_{i \in \mathcal{C}_t})$$

and only the agents contained in $\mathcal{S}_t$ are re-simulated. This formulation enables the selection that maximizes the expected total error reduction while constraining the number of agents subject to Chase to M .

B. Destination Prediction Model

We assume that the tracking system provides only time-ordered position information, which by itself is insufficient to construct the features required for prediction. Therefore, we extract the following features to construct a training dataset.

- Origin label
- Destination label
- Day of the week / time of day
- Intermediate waypoint labels
- Presence of accompanying agents
- Floor crowdedness (average number of occupants at time t)
- Coordinate sequences over the first few seconds after movement onset ($x_1, y_1, x_2, y_2, \dots$)
- Average movement speed

The classification task is formulated as a multi-class classification problem that takes all features except the destination label as input and predicts the final destination area. In this study, we construct an XGBoost classifier to perform destination prediction. Two variants of the model are constructed using coordinate sequences from the first few seconds after movement onset: a 3-s model and a 6-s model. The 3-s model is intended for use in the Lead simulation, whereas the 6-s model is designed for use in the Chase simulation.

C. Pedestrian Simulation Model

The movement simulation is implemented using Unity's NavMesh pathfinding system. NavMesh automatically discretizes walkable areas into a mesh based on the polygonal structure of the building floor plan, and simultaneously enables obstacle avoidance and inter-agent collision avoidance during path planning. Each agent is defined as a Unity NavMeshAgent component and autonomously moves toward its destination g_i .

The destination g_i corresponds to the label obtained from the destination prediction model described in the previous section, where each label is associated with a representative point (destination node) within the building. The NavMeshAgent searches for an optimal path toward this representative point and dynamically generates a path based on the predicted destination. After a path is generated, the future positions of each agent at subsequent time steps are determined along the path. The walking speed used in this process is assumed to be the average movement speed estimated from the observed data up to that time.

VI. EVALUATION

A. Experimental Environment

As shown in Fig. 3, we conduct experiments by reconstructing one floor of a university campus building in Unity. In this campus building, as illustrated in Fig. 4, human positions inside the building are tracked using point cloud data obtained from multiple LiDAR sensors, and individuals can be identified by uniquely assigned IDs. Hereafter, we refer to this system as the observation system.



Fig. 3. Reconstructed campus floor environment in Unity



Fig. 4. LiDAR sensor deployment in the indoor environment

Walkable areas are defined as a polygon mesh that includes walls and obstacles. After manual refinement, the mesh is automatically baked into a navigable area using Unity's NavMesh Surface. Each agent is instantiated as a NavMeshAgent component at the location corresponding to the coordinates obtained from the observation system. In the simulation, agents autonomously move toward the destinations predicted by the classifier. In this experiment, we define multiple OD pairs based on representative origin and destination coordinates extracted from the observed data. Each agent is assigned one of the defined OD pairs and operates independently so as not to interfere with other agents along its path. The position of each agent is sampled at 1-s intervals and exported as a text file in the format `time, id, x, y, z`. These data are treated as ground-truth data, allowing us to eliminate the effects of positioning errors and missing observations in real environments and to accurately evaluate the performance of the Lead-Chase framework in isolation.

B. Destination Prediction Classifier Evaluation

Figures 5 and 6 show the classification accuracy of the trained model, computed as the matching rate with the ground-truth labels (i.e., the actual destinations). The confusion matrices presented here represent the correspondence between

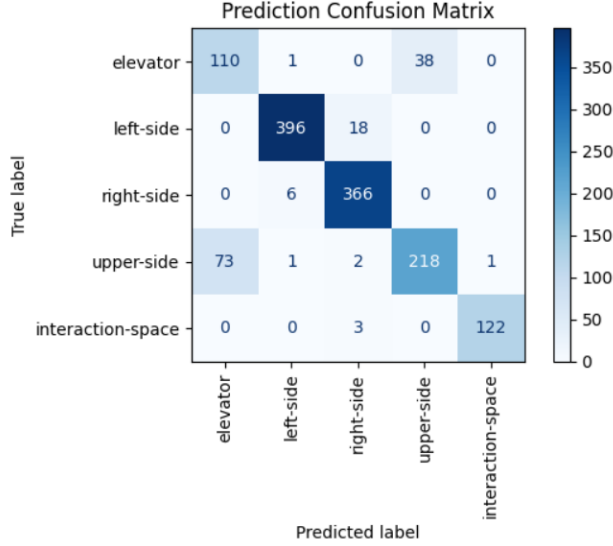


Fig. 5. Destination prediction results using a 3-second coordinate history

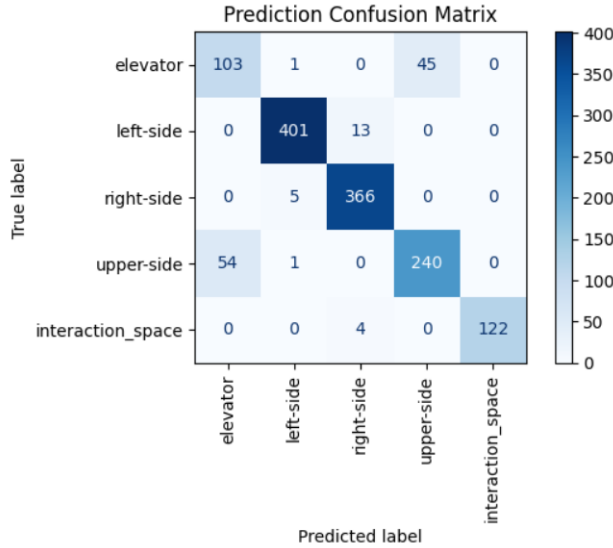


Fig. 6. Destination prediction results using a 6-second coordinate history

the true destination labels and the labels predicted by the classifier. In this study, we defined the main areas within the building (elevator, left-side, right-side, upper-side, and interaction-space) as distinct labels to uniquely identify each floor region. The average accuracy was approximately 87.2% when using a 3-second history, and approximately 90.1% when using a 6-second history, confirming that using a longer history is effective for improving destination estimation accuracy.

C. Evaluation Metrics for the Lead-Chase Framework

Prediction accuracy is evaluated using the Average Displacement Error (ADE), which represents the mean positional error over the entire prediction interval. Let $\hat{\mathbf{p}}^i(s)$ denote the

TABLE II
LEAD-CHASE ERRORS AND IMPROVEMENTS

horizon [s]	n	Lead ADE [m]	Chase ADE [m]	gain [m]
1	536	3.6638	1.1432	2.5206
2	502	4.5470	1.8655	2.6815
3	471	5.4356	2.6588	2.7768
4	436	6.3324	3.3116	3.0208
5	406	7.3258	4.0158	3.3101
6	378	8.2732	4.7521	3.5211
7	348	9.4134	5.5423	3.8711
8	319	10.3949	6.5523	3.8426
9	290	11.0810	7.4378	3.6432

predicted position of agent i at prediction time s , and let $\mathbf{p}_{\text{true}}^i(s)$ denote the corresponding ground-truth position. ADE is defined in Eq. (9).

$$\text{ADE} = \frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T \|\hat{\mathbf{p}}^i(t) - \mathbf{p}_{\text{true}}^i(t)\| \quad (9)$$

where N is the number of agents and T is the number of prediction time steps.

Table II summarizes the results. Here, *horizon* denotes the elapsed time (prediction horizon). n represents the number of agents for which both Lead and Chase predictions were successfully produced. Lead ADE and Chase ADE are the mean position errors of the Lead and Chase predictions, respectively, and *gain* denotes the improvement achieved by Chase. The gain is defined in Eq. (10).

$$\text{gain} = \text{ADE}_{\text{Lead}} - \text{ADE}_{\text{Chase}} \quad (10)$$

In this experiment, we set the observation interval to $\Delta s = 3$ [s]. Here, *horizon* denotes the elapsed time from a given time t . Suppose that the predictions in Table II are made at $t = t'$. Then, the Lead ADE reported in the table is computed from the predictions made at $t = t' - \Delta s$. In contrast, the Chase ADE is computed from the predictions made at $t = t'$. Thus, even when evaluating the same horizon, it should be noted that the timing of prediction differs between Lead and Chase.

Table II shows the position errors of Lead and Chase predictions and the corresponding improvement for each prediction horizon. First, as the horizon becomes longer, both Lead ADE and Chase ADE increase monotonically. This is likely because uncertainty grows and errors accumulate as the prediction extends further into the future. Nevertheless, Chase consistently achieves lower errors than Lead across all horizons, indicating that the error correction by Chase is effective in a robust manner. Focusing on *gain*, we observe an improvement of at least 2.5 [m] for all horizons, confirming that Chase provides sufficient correction capability. The decrease in n as time elapses indicates that fewer agents remain available for evaluation, because some agents leave the floor over time and thus disappear from the tracked population.

D. Accuracy Comparison with Lead

Chase does not re-simulate all agents; instead, it recomputes the trajectories for only a subset of agents. Nevertheless,

TABLE III
ERROR DIFFERENCE BETWEEN LEAD AND CHASE PREDICTIONS

horizon [s]	Lead error [m]	Chase error [m]	difference [m]
1	1.1765	1.6199	0.4434
2	2.2477	2.6100	0.3623
3	3.2866	3.6074	0.3208
4	4.2819	4.5864	0.3045
5	5.2346	5.4678	0.2332
6	6.1937	6.4189	0.2252
7	7.1012	7.3441	0.2429
8	8.2227	8.5207	0.2980
9	9.0771	9.4671	0.3900

as shown in Table III, the error difference from the Lead prediction (*difference*) remains at most approximately 0.44 [m] for each horizon, indicating that the overall degradation in accuracy is limited. This result suggests that selectively re-simulating only the agents that are likely to contribute most to the error can reduce computational cost while maintaining prediction accuracy.

E. Area-level Evaluation Based on People Counts

Specifically, the target space is divided into multiple functional areas, and the number of people within each area at the prediction time is aggregated. For each area a , the predicted count $\hat{N}_a$ and the ground-truth count N_a are computed, and their difference is defined as the people-count prediction error, as shown in Eq. 11

$$\Delta_a = \hat{N}_a - N_a \quad (11)$$

In addition, to evaluate the magnitude of the error, we also use $|\Delta_a|$ (absolute error). This allows us to simultaneously capture, for each area, the tendency toward overestimation or underestimation and its impact in terms of the absolute value. Furthermore, we perform the same aggregation under multiple conditions with different numbers of resimulated agents M , and visualize the area-wise people-count prediction errors to analyze how increasing or decreasing M affects the reproducibility of the spatial distribution. This evaluation approach has been used to assess the fidelity of reproducing pedestrian flows [17].

In this experiment, to evaluate the correction capability of the Lead-Chase framework, ground-truth destination labels are used for destination prediction. As shown in Fig. 7, increasing M generally reduces the magnitude of the area-wise errors and makes the predicted people counts converge toward the ground truth. This trend indicates that re-simulating a larger subset of agents improves the consistency of the predicted spatial distribution with observations, especially in high-traffic areas where local interactions strongly influence flows.

VII. CONCLUSION

In this study, we proposed a crowd digital twin platform that maintains real-time future predictions of an agent-based simulation by using sequential observation data obtained from

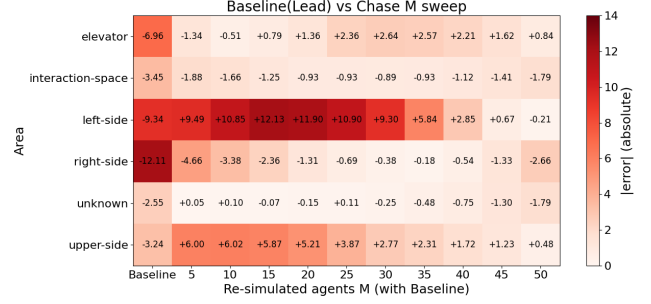


Fig. 7. Area-wise people-count error heatmaps for different M .

indoor crowd tracking. The key idea of our proposal is a Lead-Chase framework that combines Lead prediction, which forecasts future states ahead of incoming observations, with Chase prediction, which rapidly corrects the Lead predictions based on newly arrived observations. With this design, we address a real-time-specific issue—namely, that the simulation may have already finished its prediction by the time a new observation arrives, leaving no opportunity for assimilation—by avoiding full recomputation and instead restricting the correction to only the agent set $\mathcal{X}$ with large errors and the neighboring set $\mathcal{Y}$ that may be affected by them.

In our implementation, we use XGBoost for destination estimation, predicting each agent's final destination area from features such as past coordinate sequences, velocity, day-of-week and time-of-day, and crowd density. In an evaluation using data from a single floor of a campus building, we confirmed an average accuracy of approximately 87.2% when using a 3-second history from the start of movement, and approximately 90.1% when using a 6-second history, demonstrating that the destination estimation is sufficiently accurate to serve as input to the simulation. Furthermore, in the evaluation of the Lead-Chase framework, although the error increased as the prediction horizon became longer, Chase consistently outperformed Lead across all horizons, confirming that Chase yields a consistent improvement of at least 2.5 m.

In addition, for selecting agents to be included in Chase, we determine the set $\mathcal{X}$ using a threshold and formulate the selection of the remaining additional set $\mathcal{Y}$ as choosing the top candidates based on scores derived from predicted improvement. A regression model is trained using improvement values computed offline as supervisory signals, and during operation, an improvement score $\hat{g}_{t,i}$ is estimated for each candidate agent. By selecting the top $(M - |\mathcal{X}|)$ agents as $\mathcal{Y}$, we present a framework that constrains the total number of Chase targets to M while maximizing the expected error reduction. Experiments using simulation data show that, as the value of M increases, the predicted number of people in each area tends to converge toward the ground truth.

As future work, first, the proposed prediction algorithm relies on a movement model based on Unity's NavMesh, which may not sufficiently reproduce real-world pedestrian behavior. Therefore, it is necessary to improve prediction accuracy by

introducing pedestrian models with higher behavioral fidelity, such as the Social Force Model, and by identifying parameters according to the environment and crowding conditions. Second, the proposed method should be evaluated under large-scale crowd conditions, and it must be guaranteed at an operational level that the processing does not lag behind the observation arrival interval—that is, that real-time constraints are satisfied. Third, since observation noise such as localization errors, missing data, and ID switches is unavoidable in real environments, it is important to design robust assimilation and correction mechanisms that account for such noise and to conduct comprehensive evaluations under noisy conditions.

ACKNOWLEDGMENT

This work was partially funded by JST CREST Grant Number JPMJCR21M5 and JST PRESTO Grant Number JPMJPR2361.

REFERENCES

- [1] C. Chen, Y. Yang, H. Yuan, L. Chen, L. Wang, B. Qu, and D. Yang, "Animating the crowd mirage: A wifi-positioning-based crowd mobility digital twin for smart campuses," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 8, no. 4, Nov. 2024.
- [2] H. Choi, M. Fujimoto, T. Matsui, S. Misaki, and K. Yasumoto, "Wi-cal: Wifi sensing and machine learning based device-free crowd counting and localization," *IEEE Access*, vol. 10, pp. 24 395–24 410, 2022.
- [3] G. Iannizzotto, L. Lo Bello, and A. Nucita, "Passive indoor people counting by bluetooth signal deformation analysis with deep learning," *Applied Sciences*, vol. 15, no. 11, 2025.
- [4] F. Adib, Z. Kabelac, D. Katabi, and R. C. Miller, "3d tracking via body radio reflections," in *Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'14. USA: USENIX Association, 2014, p. 317–329.
- [5] Y. Hu, F. Qian, Z. Yin, Z. Li, Z. Ji, Y. Han, Q. Xu, and W. Jiang, "Experience: practical indoor localization for malls," in *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, ser. MobiCom '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 82–93.
- [6] D. Plozza, S. Marty, C. Scherrer, S. Schwartz, S. Zihlmann, and M. Magno, "Embedded 2d lidar-based person tracking for safe navigation in assistive autonomous robots," in *2025 10th International Workshop on Advances in Sensors and Interfaces (IWASI)*, 2025, pp. 1–6.
- [7] T. Li, H. Lei, H. Guo, M. Yin, Y. Hu, Q. Zhu, and S. Rangan, "Digital twin-enhanced wireless indoor navigation: Achieving efficient environment sensing with zero-shot reinforcement learning," *IEEE Open Journal of the Communications Society*, 2025.
- [8] J.-R. Lin, K.-Y. Chen, S.-Y. Song, Y.-H. Cai, P. Pan, and Y.-C. Deng, "Digital twin of buildings and occupants for emergency evacuation: Framework, technologies, applications and trends," *Advanced Engineering Informatics*, vol. 66, p. 103419, 2025.
- [9] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social lstm: Human trajectory prediction in crowded spaces," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.
- [10] J. Ngiam, B. Caine, V. Vasudevan, Z. Zhang, H.-T. L. Chiang, J. Ling, R. Roelofs, A. Bewley, C. Liu, A. Venugopal, D. Weiss, B. Sapp, Z. Chen, and J. Shlens, "Scene transformer: A unified architecture for predicting multiple agent trajectories," 2022. [Online]. Available: <https://arxiv.org/abs/2106.08417>
- [11] K. Jain, S. Prawesh, I. S. Dalal, and A. Tripathi, "Benchmarking pedestrian dynamics models for common scenarios: An evaluation of force-based models," 2025. [Online]. Available: <https://arxiv.org/abs/2501.05106>
- [12] A. Giese, D. Latypov, and N. M. Amato, "Reciprocally-rotating velocity obstacles," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 3234–3241.
- [13] R. Clay, L.-M. Kieu, J. A. Ward, A. Heppenstall, and N. Malleon, "Towards real-time crowd simulation under uncertainty using an agent-based model and an unscented kalman filter," in *Advances in Practical Applications of Agents, Multi-Agent Systems, and Trustworthiness. The PAAMS Collection*, Y. Demazeau, T. Holvoet, J. M. Corchado, and S. Costantini, Eds. Cham: Springer International Publishing, 2020, pp. 68–79.
- [14] F. Makinoshima, T. Takahashi, and Y. Oishi, "Bayesian behavioural model estimation for live crowd simulation," in *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS '24. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2024, p. 1355–1362.
- [15] N. Malleon, K. Minors, L.-M. Kieu, J. Ward, A. West, and A. Heppenstall, "Simulating crowds in real time with agent-based modelling and a particle filter," *Journal of Artificial Societies and Social Simulation*, vol. 23, no. 3, pp. 1–3, None 2020.
- [16] H. Yamaguchi, H. Rizk, T. Amano, A. Hiromori, R. Ukyo, S. Yamada, and M. Ohno, "Towards intelligent environments: human sensing through 3d point cloud," *Journal of Reliable Intelligent Environments*, vol. 10, pp. 281–298, 08 2024.
- [17] U. Masashi, T. Amano, and H. Yamaguchi, "Simulating urban pedestrian flows by fusing wide-area location data and spot pedestrian counts," in *Mobile and Ubiquitous Systems: Computing, Networking and Services*, A. Soylu, F. Liu, K. Mitra, Y. Zhang, and T.-M. Grønli, Eds. Cham: Springer Nature Switzerland, 2026, pp. 550–569.